

Machine Learning Workshop | L05

Computer Vision: Worked example | AI safety

Where we are so far

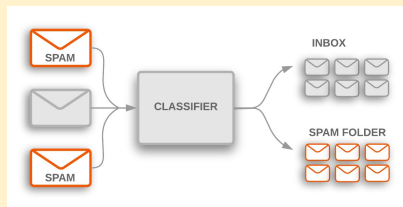
Artificial Intelligence

Any techniques that enables computers to mimic human behaviour



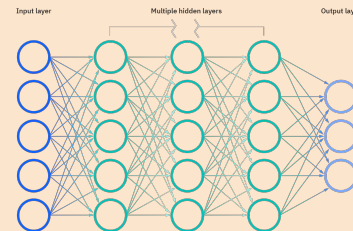
Machine Learning

Ability to learn without explicitly being programmed

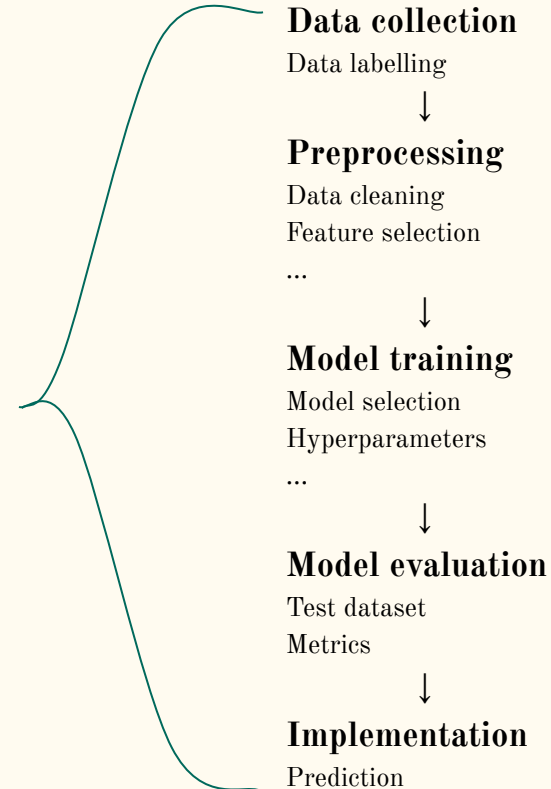
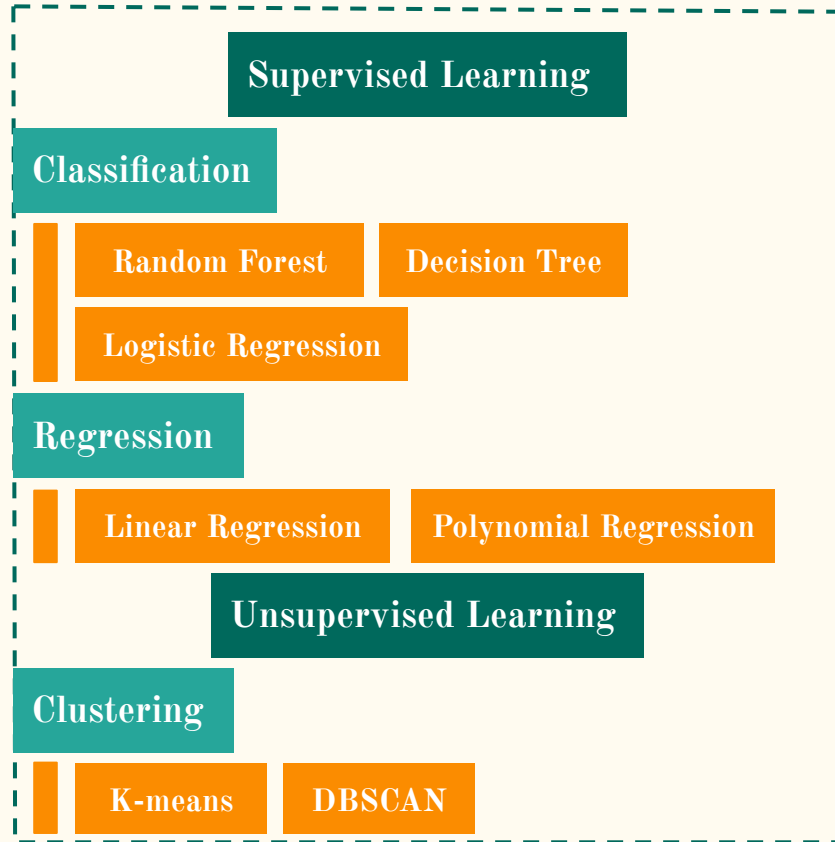


Deep Learning

Extract patterns from data using neural networks



Recap - Machine learning approach



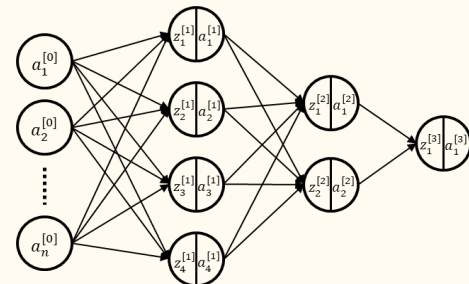
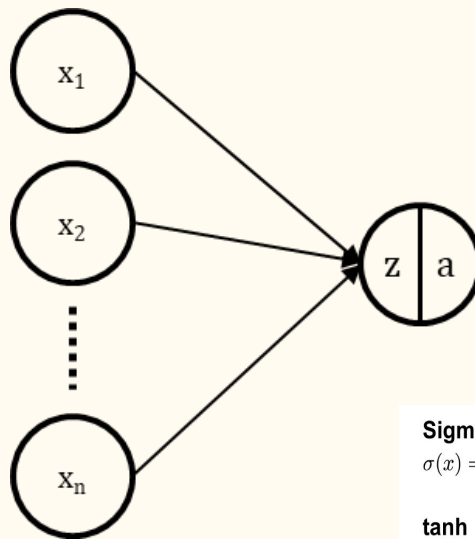
Recap - From regression to neural network

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = y$$

Multiple regression

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

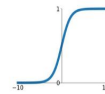
Neural network



Activation functions

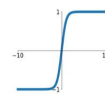
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



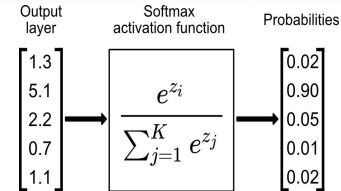
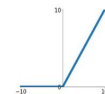
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$



Recap - Loss functions

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

Loss Function $\rightarrow$ try different w / b $\rightarrow$ minimize the loss function

How do we update the parameters from results : Backpropagation

Forward Pass $\rightarrow$

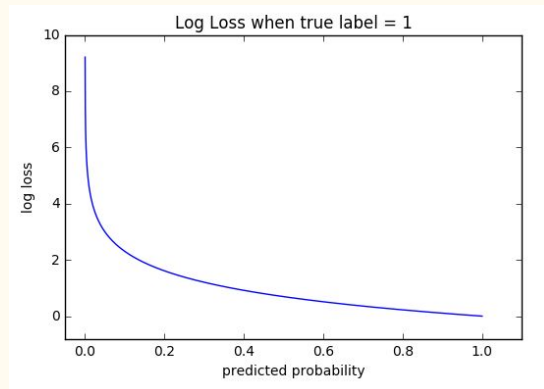


Back propagation

- Gradient Descent
- Update parameters

Cross-entropy loss

$$L(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$



Recap - Gradient Descent

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

Loss function is the difference between predicted class and actual class.

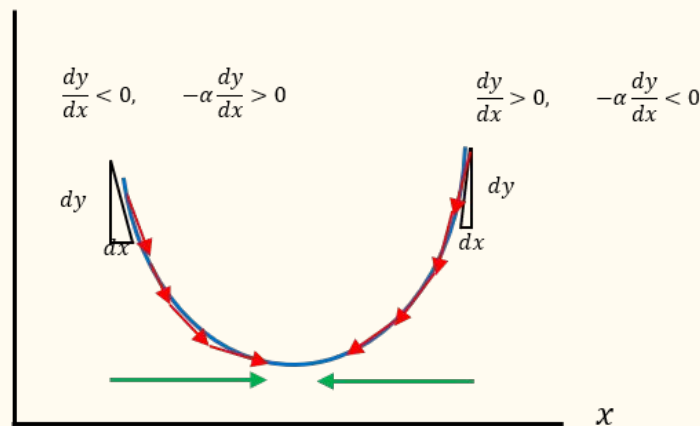
Aim : adjust the **weight** and **bias** until the loss function is **minimum**.

Method : Take gradient of loss function $\rightarrow$ set it to 0 $\rightarrow$ tune parameter b, w

1. Take Gradient of the Loss Function
2. Pick random values for the parameters
3. Plug the parameters value into the gradient
4. Step sizes : step size = slope x learning rate
5. New parameters = Old parameter - step size

| **Chain rule** in backpropagation

$$y = f(x)$$



Recap - Activation functions

Gradient vanishing : gradient getting smaller and smaller in BP $\rightarrow$ tends to 0

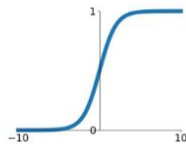
Gradient exploding : gradient getting greater and greater in BP $\rightarrow$ weight diverges

binary clf: sigmoid

multi-class clf: softmax

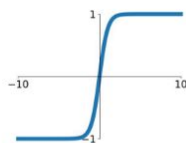
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



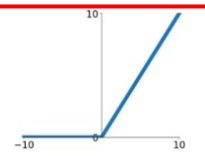
tanh

$$\tanh(x)$$

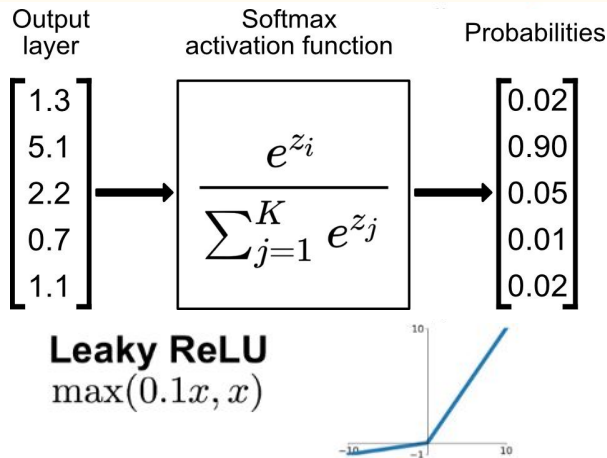


ReLU

$$\max(0, x)$$



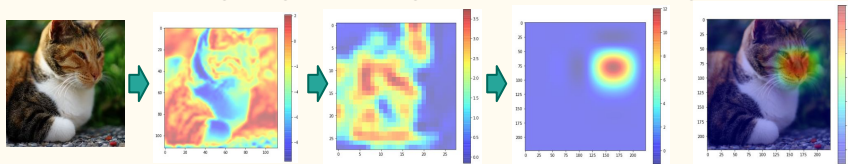
ReLU is a good default
choice for most problems



Recap - Feature learning

- ❖ Feature extractor (Conv_layer) is learned by computer itself
- ❖ Uses multiple layers to progressively extract higher-level features from raw input (e.g. image)
- ❖ Higher-level features and concepts are defined in terms of lower-level ones
- ❖ “Hierarchical” Feature Learning
 - **Deep Learning**

Lower-level features (e.g. edges) $\Rightarrow$ Higher-level features (e.g. cat nose/ whiskers)



Recap - Paradigm shift of machine learning

❖ Big Data

- Volume of data & storage & flow
- Cannot use traditional way to store and analysis the data
- e.g. Large-scale dataset: ImageNet



❖ Deep Learning

- “Hierarchical” Feature Learning
 - Convolutional Neural Network

❖ Hardware and software

- GPU Graphics Processing Unit

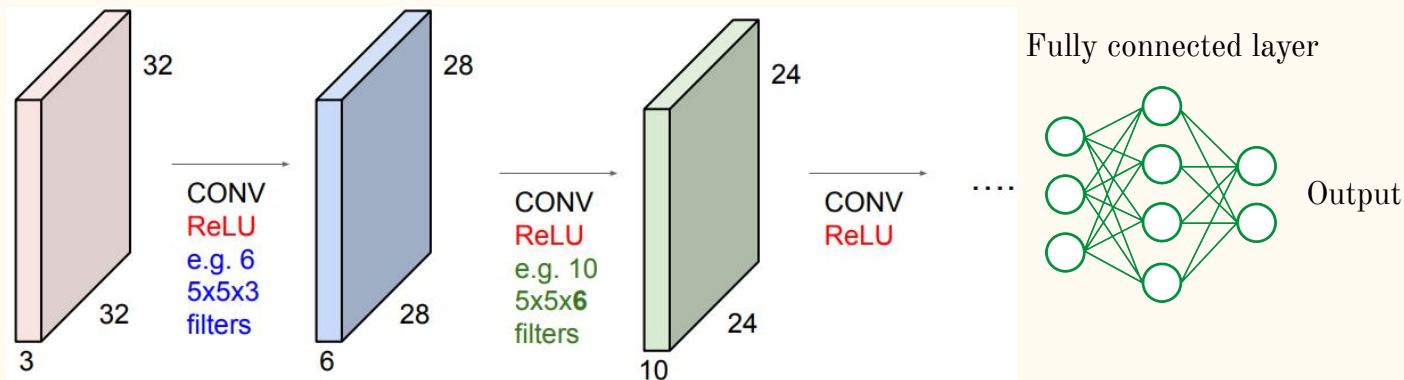
AlexNet

SuperVision	<u>Alex Krizhevsky,</u> Ilya Sutskever, Geoffrey Hinton University of Toronto	Our model is a large, <u>deep convolutional neural network</u> trained on raw RGB pixel values. The neural network, which has 60 million parameters and <u>650,000 neurons</u>, consists of five convolutional layers, some of which are followed by max-pooling layers, and <u>three globally-connected layers</u> with a final <u>1000-way softmax</u>. It was trained on <u>two NVIDIA GPUs for about a week</u>. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of convolutional nets. To reduce overfitting in the globally-connected layers we employed hidden-unit "dropout", a recently-developed regularization method that proved to be very effective.
-------------	---	---

Recap - Convolutional Neural Network

Convolutional Neural Network (CNN) is a sequence of convolution layers, interspersed with activation functions.

- **Key:** To filter the information and extract features from input data



Case study: AlexNet

Worked Example:

Weather Classification

Weather classification

Aims: Construct a deep learning model to classify different weather conditions

Given 4 classes:



[Kaggle] Labeled dataset:

```
class Sunrise contain 287 train images and 70 test images
class Cloudy contain 240 train images and 60 test images
class Rainy contain 173 train images and 42 test images
class Sunny contain 203 train images and 50 test images
```

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

Train_validation_test split

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

Data Cleaning (missing value, noisy data, ...) for quality/robust results.

Validation set will be used to tune the hyperparameter.

Test set is not touched until after model is finalized.



Recap - Why normalisation ?

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

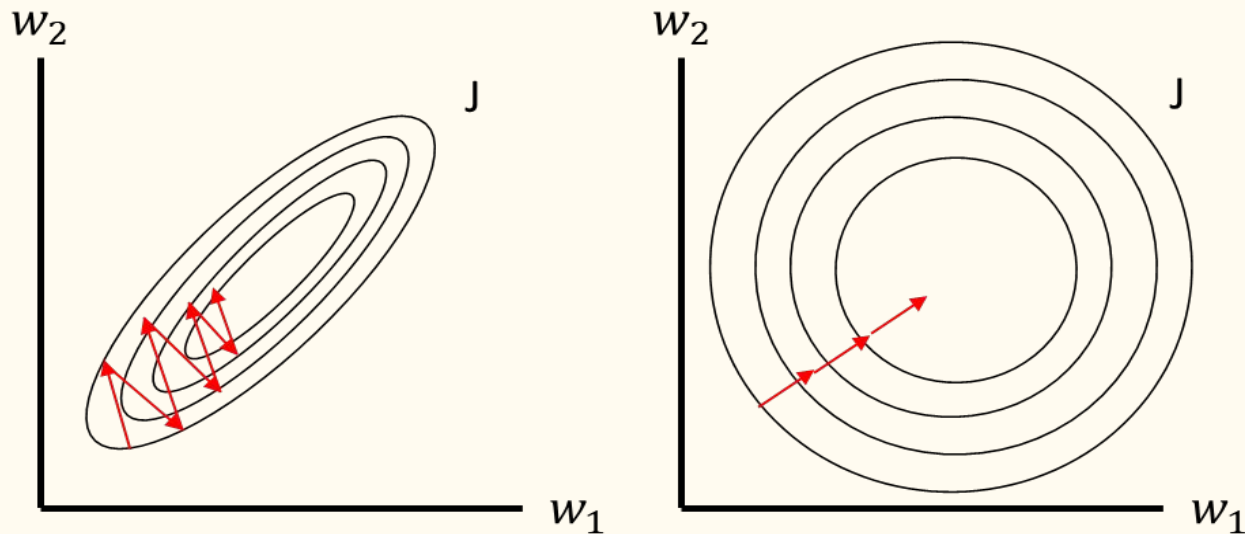
Test dataset

Metrics



Implementation

Prediction



Building model from scratch

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

Under Keras/Tensoflow framework

MaxPooling2D class

```
tf.keras.layers.MaxPooling2D(  
    pool_size=(2, 2), strides=None, padding="valid", data_format=None, **kwargs  
)
```

<https://keras.io/api/layers/>

Conv2D class

```
tf.keras.layers.Conv2D(  
    filters,  
    kernel_size,  
    strides=(1, 1),  
    padding="valid",  
    data_format=None,  
    dilation_rate=(1, 1),  
    groups=1,  
    activation=None,  
    use_bias=True,  
    kernel_initializer="glorot_uniform",  
    bias_initializer="zeros",  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    **kwargs  
)
```

Hyper-parameters

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

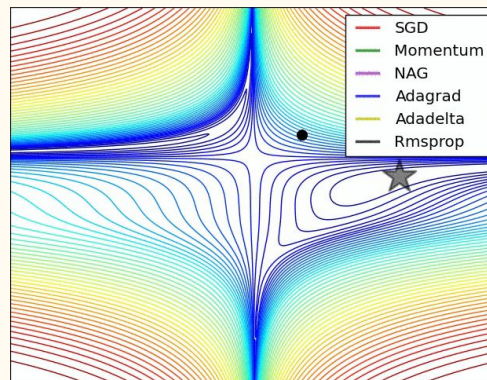
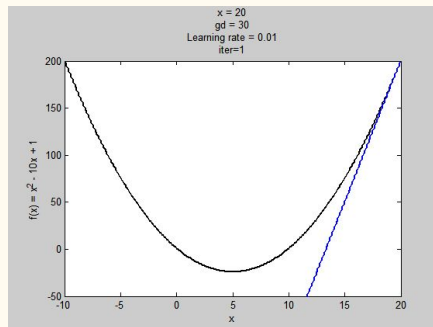
Prediction

Optimizer - the way you hiking down the loss function “hill”

❖ Stochastic Gradient Descent (SGD)

➤ Mini-batch Gradient Descent

➤ Batch size, step, epoch



Batch - Step - Epoch

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

A training step is one gradient update. In one step, batch size of images/data are processed. After one epoch, the model processed one full cycle of training data.

For example, there are 500 images, if batch size is 10, then 10 images are processed in one step. For the model processes all training data once(=one epoch), it takes 50 steps.

Batch - Step - Epoch

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

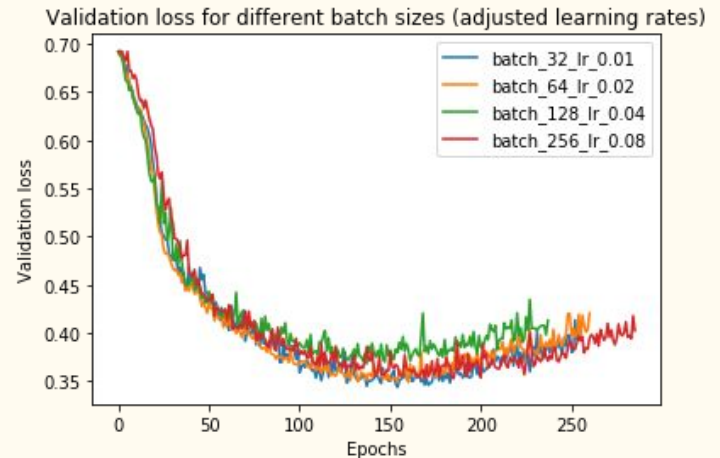
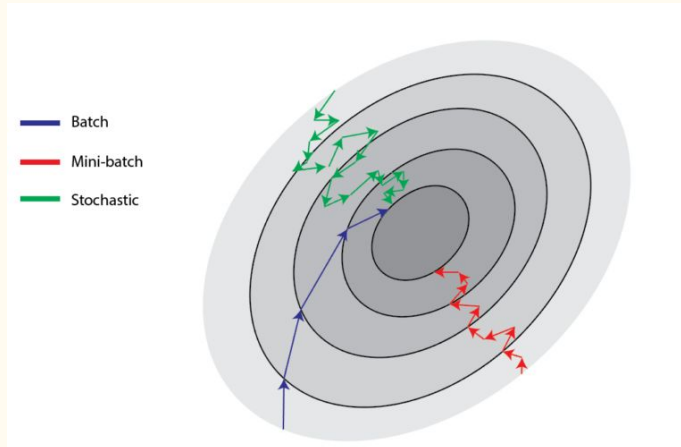
Prediction

Three gradient descend variants:

Stochastic: using one data value in one gradient update

Mini-batch: using certain number of data in one gradient update

Batch: using all training data at once



Batch - Step - Epoch

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

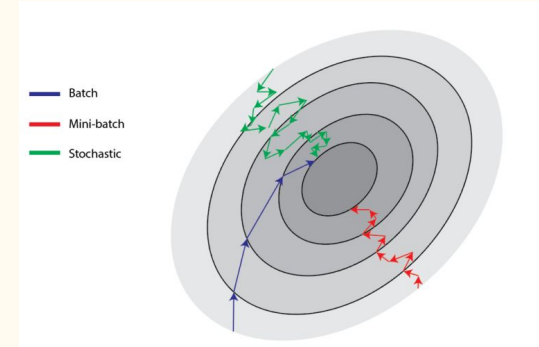
Prediction

Three gradient descend variants:

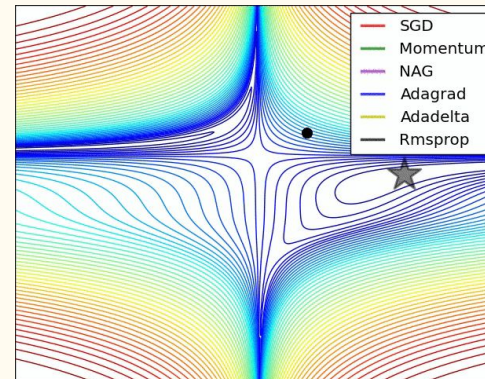
Stochastic: using one data value in one gradient update

Mini-batch: using certain number of data in one gradient update

Batch: using all training data at once



There are many gradient descent optimizers



Calculation: parameters and output dims

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

Parameters = number of weight + number of Bias

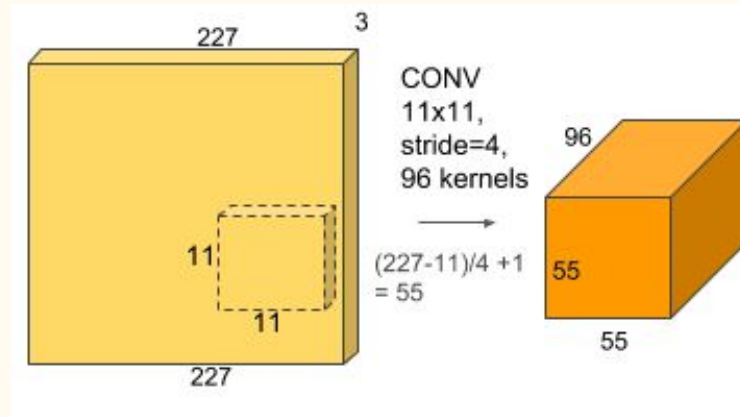
Output size:
$$\text{floor}\left(\frac{W + 2 \times \text{pad} - ks}{S}\right) + 1$$

W: input size

pad: padding

ks: kernel size

S: stride



Training dynamic - callbacks

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

A callback is a set of functions to be applied at given stages of the training procedure.

- EarlyStopping - stop the model when it's no longer improving

- the “brake” in a car



- ModelCheckpoint - saves the model after every epoch
- LearningRateScheduler - adjusts the learning rate over time

Training dynamic - Loss and accuracy

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

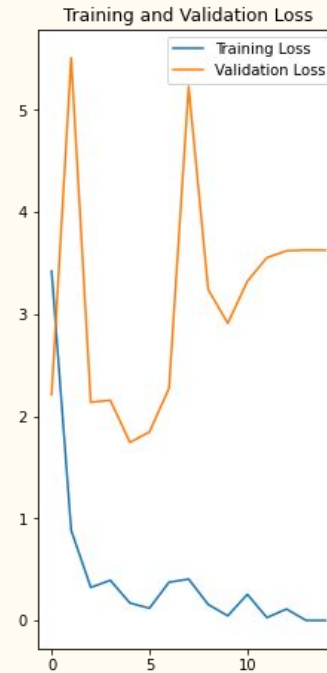
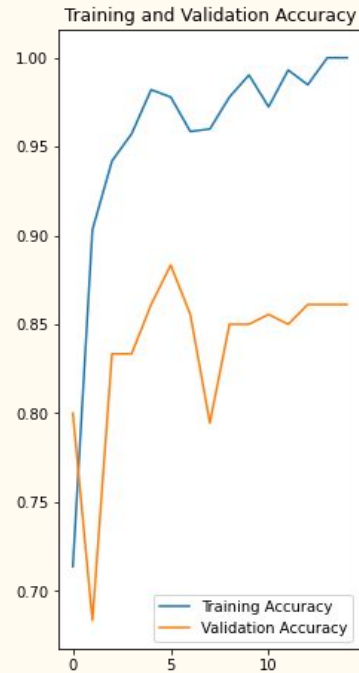
Test dataset

Metrics



Implementation

Prediction



Transfer Learning

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics

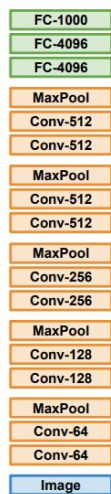


Implementation

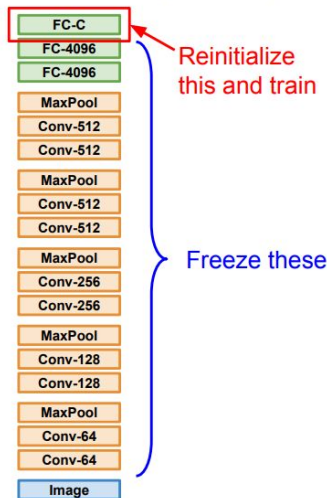
Prediction

Trainable params vs Non-trainable params

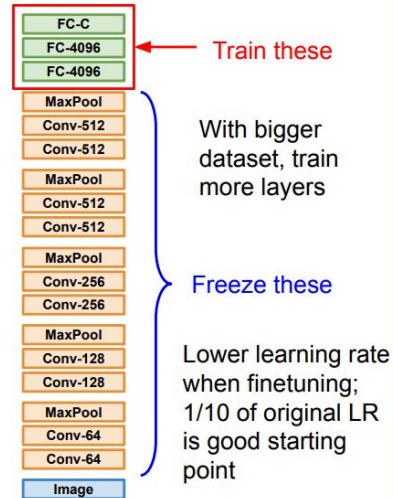
1. Train on Imagenet



2. Small Dataset (C classes)

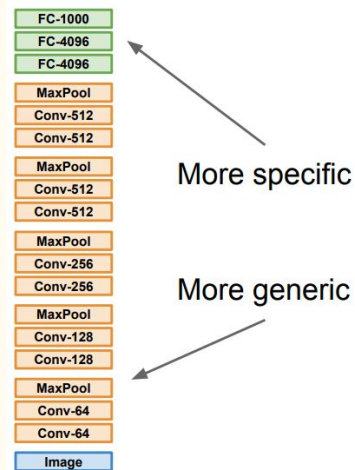


3. Bigger dataset



=====

Total params: 2,585,668
Trainable params: 327,684
Non-trainable params: 2,257,984



Pre-trained model

Model efficiency

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics

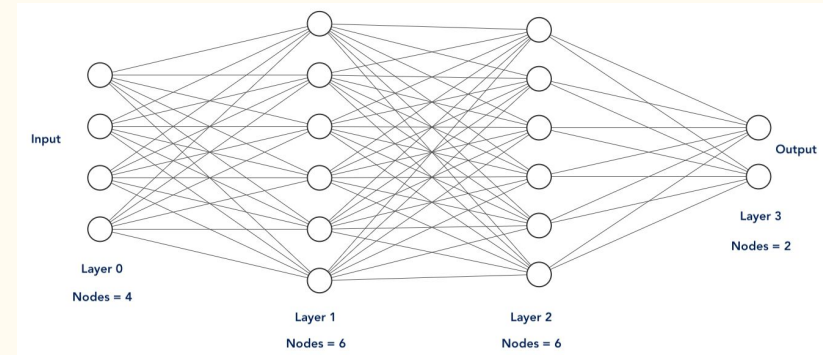


Implementation

Prediction

Trade-off in speed, accuracy

- Number of parameters
- Width (Number of neurons)
- Depth (Number of layers)



[EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks](#)

[Efficient Deep Learning: A Survey on Making Deep Learning Models Smaller, Faster, and Better](#)

Evaluation - Metrics

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

In classification context,

- Confusion matrix
- Accuracy
- Precision
- Recall
- F1-score
- ...

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{All Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

		Predicted classes	
		Positive	Negative
Actual classes	Positive	TP	FN
	Negative	FP	TN

Evaluation - Metrics

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

In this case, the model accuracy

$$= (5+1000) / (1000+5+5+10)$$

$$= 98.5\%$$

Problem: if it is positive, then the model has
50% chance of detecting it as negative.

⇒ Precision, recall, and F1-score

$$Accuracy = \frac{\text{Correct Predictions}}{\text{All Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Unbalanced dataset!

		Predicted classes	
		Positive	Negative
Actual classes	Positive	5	5
	Negative	10	1000

Evaluation - Precision & Recall

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

Precision: a measure of quality

Recall: a measure of quantity

In this case, **accuracy** = 98.5%

Precision of class “Positive” = $5 / (5 + 10) = 33.3\%$

Recall of class “Positive” = $5 / (5 + 5) = 50\%$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Accuracy = \frac{\text{Correct Predictions}}{\text{All Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

		Predicted classes	
		Positive	Negative
Actual classes	Positive	5 (TP)	5 (FN)
	Negative	10 (FP)	1000 (TN)

Evaluation - Confidence threshold

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Confidence threshold - to determine whether or not the prediction is the final decision.

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

confidence threshold: 60%

Actual Classes	Predicted Classes	Final Decision
P	P (72%)	P
P	P (82%)	P
P	P (55%)	N
N	P (33%)	N
N	P (65%)	P
N	P (21%)	N

Accuracy
= 66.6%

Precision(P)
= 2/3
= 66.6%

Recall(P)
= 2/3
= 66.6%

TP

TP

FN

TN

FP

TN

confidence threshold: 75%

Actual Classes	Predicted Classes	Final Decision
P	P (72%)	N
P	P (82%)	P
P	P (55%)	N
N	P (33%)	N
N	P (65%)	N
N	P (21%)	N

Accuracy
= 50%

Precision(P)
= 1/1
= 100%

Recall(P)
= 1/3
= 33.3%

FN

TP

FN

TN

TN

TN

Evaluation - Confidence threshold

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Confidence threshold - to determine whether or not the prediction is the final decision.

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

confidence threshold: 60%

Actual Classes	Predicted Classes	Final Decision
P	P (72%)	P
P	P (82%)	P
P	P (55%)	N
N	P (33%)	N
N	P (65%)	P
N	P (21%)	N

TP

Accuracy
= 66.6%

TP

Precision(P)

= 2/3
= 66.6%

TN

Recall(P)
= 2/3

= 66.6%

TN

Precision-recall tradeoff!

confidence threshold: 75%

Actual Classes	Predicted Classes	Final Decision
P	P (72%)	N
P	P (82%)	P
P	P (55%)	N
N	P (33%)	N
N	P (65%)	N
N	P (21%)	N

FN

Accuracy
= 50%

TP

Precision(P)

= 1/1
= 100%

TN

Recall(P)
= 1/3

= 33.3%

TN

Evaluation - Precision-Recall curve

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

Precision: a measure of quality

Recall: a measure of quantity

Precision-recall tradeoff:

Increasing confidence threshold $\Rightarrow$ higher precision but lower recall

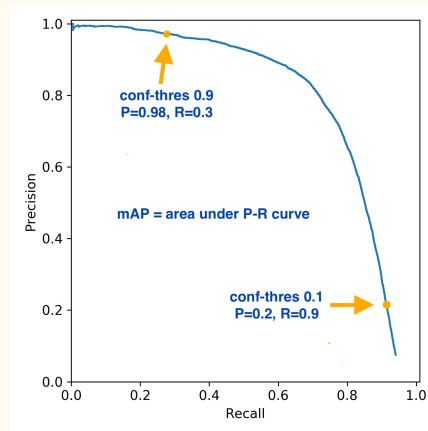
Evaluate the model with different conf-thres

$\Rightarrow$ gives Precision-Recall Curve PRC

$$Accuracy = \frac{\text{Correct Predictions}}{\text{All Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$



Evaluation - Precision & Recall & F1-score

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

Precision: a measure of quality

Recall: a measure of quantity

Precision-recall tradeoff:

Increasing confidence threshold $\Rightarrow$ higher precision but lower recall

Evaluate the model with different conf-thres

$\Rightarrow$ gives Precision-Recall Curve PRC

F1-score: harmonic mean of precision and recall

usually use f1-score to represent the performance of model at certain confidence threshold

$$Accuracy = \frac{\text{Correct Predictions}}{\text{All Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2}{recall^{-1} + precision^{-1}}$$

Hyper-parameters optimization

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

Prediction

- Data quality, data preprocessing
- Model architecture (scaling): parameters, width, depth
- Optimizers: batch size, learning rate
- Activation functions
- Regularization: dropout, ...
- ...

<https://towardsdatascience.com/17-rules-of-thumb-for-building-a-neural-network-93356f9930af>

Evaluation - Feature map

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



Implementation

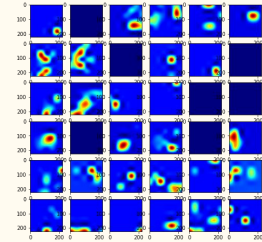
Prediction

Reasoning - ask **why** the computer makes such a prediction.

Feature map:

visualize where the computer is looking on the image, and

understand what is the determining factor of prediction



Weight input value bias

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$

Activation fn Feature value/ output

$$g(z) = a$$

From Deep Learning to Deep Reasoning

Authors: Triet Tran Vuoc Le Hung Le Thanh M. Le [Authors info & Claims](#)

KDD '21: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining • August 2021 • Pages 4076–4077 • <https://doi.org/10.1145/3447548.3470803>

Online: 14 August 2021 [Publication History](#)

11 1 299

Get Access

ABSTRACT

The rise of big data and big compute has brought modern neural networks to many walks of digital life, thanks to the relative ease of constructing large models that scale to the real world. Current successes of Transformers and self-supervised pretraining on massive data have led some to believe that deep neural networks will be able to do almost everything once we have sufficient data and computational resources. However, neural networks are fast to exploit surface statistics but fail miserably to generalize to novel combinations. This is because they are not designed for deliberate reasoning – the capacity to deliberately deduce new knowledge out of the contextualized data. This tutorial reviews recent developments to extend the capacity of neural

Evaluation - Feature map

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics

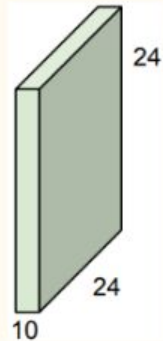


Implementation

Prediction

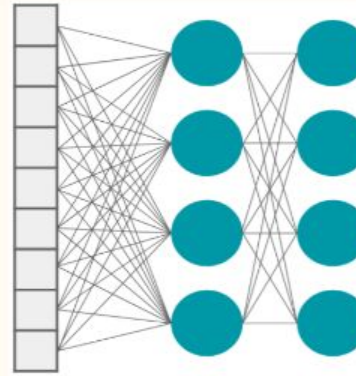
Reasoning - ask **why** the computer makes such a prediction.

the last pooled feature map



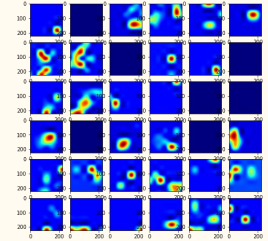
Flattening

one-dims vector
(contain 5760 values)



Fully connected layer

Output



Implemenetation - make a prediciton

Data collection

Data labelling



Preprocessing

Data cleaning

...



Model training

Model selection

Hyperparameters

...



Model evaluation

Test dataset

Metrics



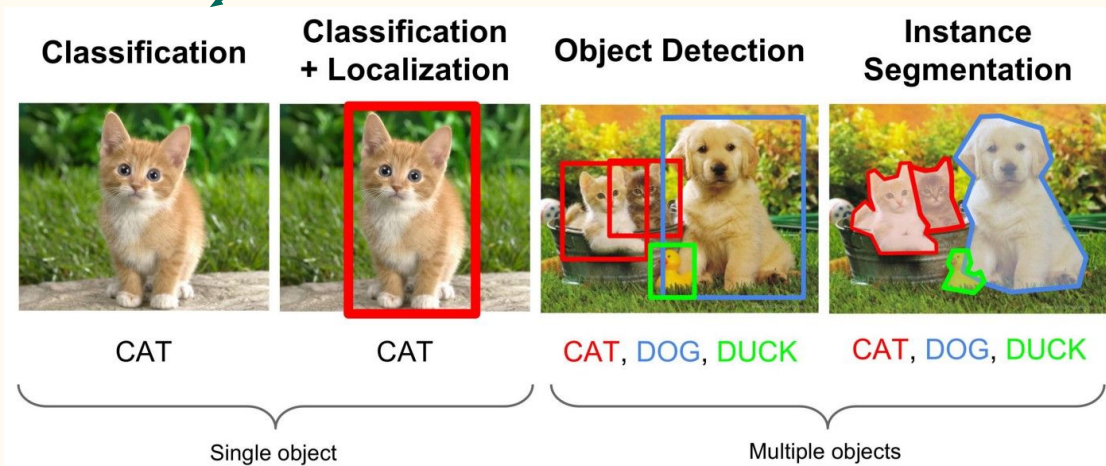
Implementation

Prediction

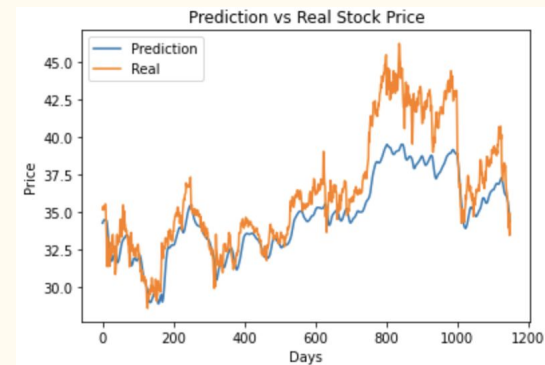
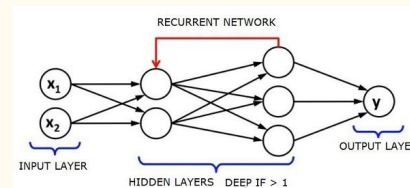
1. Try and do prediction on your own image
2. Evaluate your result

More...

where we are: Multi-class classification



RNN



More...

Toward a Climatology of Fog Frequency in the Atacama Desert via Multispectral Satellite Data and Machine Learning Techniques

Christoph Böhm, Jan H. Schween, Mark Reyers, Benedikt Maier, Ulrich Löhnert, and Susanne Crewell

transfer simulations for a limited amount of frequency bands, the aim of this study is to explore the entire information available from

spaceborne sensors. To capture different, yet presumably distinct, radiative signatures that are characteristic scenes, a **machine learning** technique is applied to recognize the relevant patterns. **Machine learning** techniques in remote sensing and earth observation (e

... [Show More](#)

Journal: *Journal of Applied Meteorology and Climatology*

DOI: <https://doi.org/10.1175/JAMC-D-20-01001>

Volume/Issue: Volume 60: Issue 8

Published Online: 13 Aug 2021

[Abstract](#)

[Download PDF](#)

Automatic cloud classification of whole sky images

A. Heinle¹, A. Macke², and A. Srivastav¹

¹Excellence Cluster "The Future Ocean", Department of Computer Science, Kiel University, Kiel, Germany

²Leibniz Institute of Marine Sciences at Kiel University (IfM-GEOMAR), Kiel, Germany

Received: 21 Oct 2009 – Discussion started: 27 Jan 2010 – Revised: 09 Apr 2010 – Accepted: 20 Apr 2010 – Published: 06 May 2010

Abstract. The recently increasing development of whole sky imagers enables temporal and spatial high-resolution sky observations. One application already performed in most cases is the estimation of fractional sky cover. A distinction between different cloud types, however, is still in progress. Here, an automatic cloud classification algorithm is presented, based on a set of mainly statistical features describing the color as well as the texture of an image. The k-nearest-neighbour classifier is used due to its high performance in solving complex issues, simplicity of implementation and low computational complexity. Seven different sky conditions are distinguished: high thin clouds (cirrus and cirrostratus), high patched cumuliiform clouds (cirrocumulus and altocumulus), stratocumulus clouds, low cumuliiform clouds, thick clouds (cumulonimbus and nimbostratus), stratiform clouds and clear sky. Based on the Leave-One-Out Cross-Validation the algorithm achieves an accuracy of about 97%. In addition, a test run of random images is presented, still outperforming previous algorithms by yielding a success rate of about 75%, or up to 88% if only "serious" errors with respect to radiation impact are considered. Reasons for the decrement in accuracy are discussed, and ideas to further improve the classification results, especially in problematic cases, are investigated.

How to cite. Heinle, A., Macke, A., and Srivastav, A.: Automatic cloud classification of whole sky images, *Atmos. Meas. Tech.*, 3, 557–567, <https://doi.org/10.5194/amt-3-557-2010>, 2010.

Development of a Method to Detect High Ice Water Content Environments Using Machine Learning

Julie A. Haggerty, Allyson Rugg, Rodney Potts, Alain Protat, J. Walter Strapp, Thomas Ratvasky, Kristopher Bedka, and Alice Grandin

(NWP) models, satellite cloud retrievals, and ground-based radar products. An artificial intelligence approach based on fuzzy-logic technology is implemented to combine information derived from each data source. Research aircraft measurements acquired from field experiments are used to objectively train the algorithm using a **machine-learning** technique. Section 2 explains the approach upon which ALPHA is based. Observations used to train and verify the method are presented in section 3

Deep Learning on Three-Dimensional Multiscale Data for Next-Hour Tornado Prediction

Journal: *Journal of Applied Meteorology and Climatology*
Volume/Issue: Ryan Lagerquist, Amy McGovern, Cameron R. Homeyer, David John Gagne II, and Travis Smith

Abstract. (Cintineo et al. 2018). Convolutional neural networks (CNN) are specially designed to learn from spatial grids and often contain many layers, which qualifies them as a **deep-learning** method (section 1.1.4 of Chollet 2018). In traditional ML, spatial grids must be transformed into scalar features, which become the direct inputs to the model. Examples are principal components, spatial statistics (such as means and standard deviations), and raw gridpoint values (where each value in the g

... [Show More](#)

Journal: *Monthly Weather Review*

DOI: <https://doi.org/10.1175/MWR-D-19-0372.1>

Volume/Issue: Volume 148: Issue 7

Published Online: 24 Jun 2020

[Abstract](#)

[Download PDF](#)

AI safety

1. Information security

DESCENDANTS
OF THE ABACUS

SITCON 2021
失控的深度學習模型
從影像、文字到語音

General adversarial attack against Deep Learning
model in image, text, and audio domain

2021/09/04
M.Y. Ho (Kaminyou)
ikaminyou@gmail.com

Presentation
失控的深度學習模型：
從影像、文字到語音
— 何明洋 / Kaminyou

DESCENDANTS
算 盤 後 裔
OF THE ABACUS

SITCON Akatsuki Taiwan Panasonic AMD

[SITCON 2021] R1 失控的深度學習模型：從影像、文字到語音

[失控的深度學習模型:從影像、文字到語音]

Can computer intentionally make a wrong prediction?



Sure!

$$y = f(x_1, x_2, x_3, \dots)$$

$$\text{softmax}(\text{output}) = f(\text{feature.values})$$

$$[0.1, 0.7, 0.2] = f\left(\left[\begin{array}{c} \text{STOP} \end{array}\right]\right) = f\left(\begin{bmatrix} 211 & 245 \\ 201 & 256 \end{bmatrix}\right)$$

SpeedLimit60

STOP

...

Activation fn

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

Feature value/ output

$$\mathbf{z}^{[1]} = \mathbf{w}^{[1]}\mathbf{a}^{[0]} + \mathbf{b}^{[1]}$$

$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & \dots & w_{1n}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & \dots & w_{2n}^{[1]} \\ \vdots & \vdots & \dots & \vdots \\ w_{41}^{[1]} & w_{42}^{[1]} & \dots & w_{4n}^{[1]} \end{bmatrix} \times \begin{bmatrix} a_1^{[0]} \\ a_2^{[0]} \\ \vdots \\ a_n^{[0]} \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

4×1 $4 \times n$ $n \times 1$ 4×1

Sure! method 1: update parameters

$$y = f(x_1, x_2, x_3, \dots)$$

$$y = ax^2 + bx + c$$



Generate adversarial target y

If $y = 10$, the prediction will be STOP

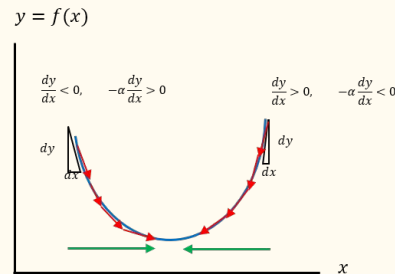
If $y = 19$, the prediction will be SpeedLimit60

Want $x=1$, $y=19$, but now $y = 10$

$$10 = 2 \times 1^2 + 3 \times 1 + 5$$

After updating parameters, $y = 19$!

$$19 = 6 \times 1^2 + 7 \times 1 + 6$$



Sure! method 2: modify input data/pixel



$$y = f(x_1, x_2, x_3, \dots)$$

$$y = ax^2 + bx + c$$

Generate adversarial target y

If $y = 10$, the prediction will be STOP

If $y = 19$, the prediction will be SpeedLimit60

Want $y=19$, but now $y = 10$

$$10 = 2 \times 1^2 + 3 \times 1 + 5$$

After changing $x = 2$, $y = 19$!

$$19 = 2 \times 2^2 + 3 \times 2 + 5$$

Sure! method 2: modify input data/pixel



$y = \text{"koala bear"}$

+



Noise

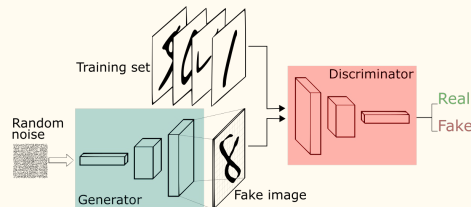
$\Rightarrow$



$y = \text{"Horse"}$

Attack and Defense

1. Adversarial training (e.g GANs)



2. Train with augmented data/image (e.g. adding noise to training data)

3. Hide your model architecture

4. ...



2. Blackbox nature of deep learning model

Lack of understanding of how it works or where the prediction comes from

$$z_1^{[1]} = w_{11}^{[1]} a_1^{[0]} + w_{12}^{[1]} a_2^{[0]} + \dots + w_{1n}^{[1]} a_n^{[0]} + b_1^{[1]}$$

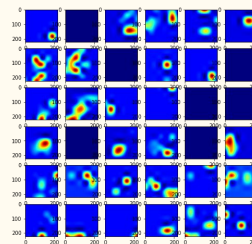
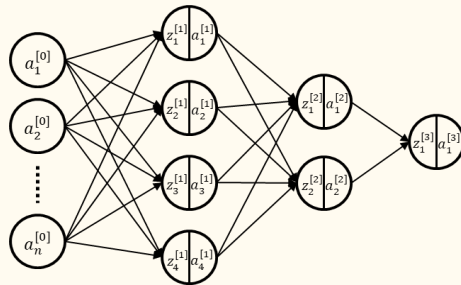
$$a_1^{[1]} = g(z_1^{[1]})$$

$$z_2^{[1]} = w_{21}^{[1]} a_1^{[0]} + w_{22}^{[1]} a_2^{[0]} + \dots + w_{2n}^{[1]} a_n^{[0]} + b_2^{[1]}$$

$$z_1^{[2]} = w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + w_{13}^{[2]} a_3^{[1]} + w_{14}^{[2]} a_4^{[1]} + b_1^{[2]}$$

$$z_{11}^{[3]} = w_{11}^{[3]} a_1^{[2]} + w_{21}^{[3]} a_2^{[2]} + b_1^{[3]}$$

$$a_1^{[3]} = g(z_1^{[3]}) = \hat{y}$$



AlexNet will generate **256** feature maps in the last convolution layer

```
max_pooling2d_20 (MaxPoolin (None, 6, 6, 256)
g2D)

flatten_6 (Flatten) (None, 9216)

dense_16 (Dense) (None, 4096)

dropout_6 (Dropout) (None, 4096)

dense_17 (Dense) (None, 4096)

dropout_7 (Dropout) (None, 4096)

dense_18 (Dense) (None, 4)

=====
Total params: 58,303,236
Trainable params: 58,300,484
Non-trainable params: 2,752
```

2. Blackbox nature of deep learning model

Lack of understanding of how it works or where the prediction comes from

$$z_1^{[1]} = w_{11}^{[1]} a_1^{[0]} + w_{12}^{[1]} a_2^{[0]} + \dots + w_{1n}^{[1]} a_n^{[0]} + b_1^{[1]}$$

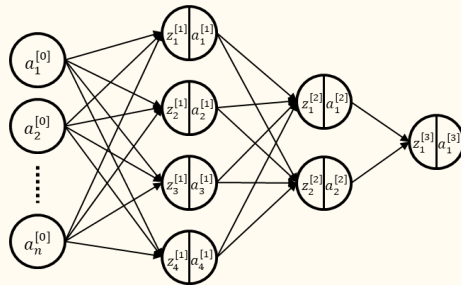
$$a_1^{[1]} = g(z_1^{[1]})$$

$$z_2^{[1]} = w_{21}^{[1]} a_1^{[0]} + w_{22}^{[1]} a_2^{[0]} + \dots + w_{2n}^{[1]} a_n^{[0]} + b_2^{[1]}$$

$$z_1^{[2]} = w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + w_{13}^{[2]} a_3^{[1]} + w_{14}^{[2]} a_4^{[1]} + b_1^{[2]}$$

$$z_{11}^{[3]} = w_{11}^{[3]} a_1^{[2]} + w_{21}^{[3]} a_2^{[2]} + b_1^{[3]}$$

$$a_1^{[3]} = g(z_1^{[3]}) = \hat{y}$$



[“Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead”](#)

[“Unpredictability of AI”](#)

Perspective | Published: 13 May 2019

Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead

Cynthia Rudin

Nature Machine Intelligence 1, 206–215 (2019) | [Cite this article](#)

53k Accesses | 1331 Citations | 460 Altmetric | [Metrics](#)

1 A preprint version of the article is available at arXiv.

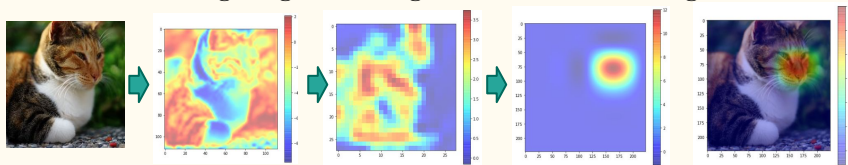
Abstract

Black box machine learning models are currently being used for high-stakes decision making throughout society, causing problems in healthcare, criminal justice and other domains. Some people hope that creating methods for explaining these black box models will alleviate some of the problems, but trying to explain black box models, rather than creating models that are interpretable in the first place, is likely to perpetuate bad practice and can potentially cause great harm to society. The way forward is to design models that are inherently interpretable. This Perspective clarifies the chasm between explaining black boxes and using inherently interpretable models, outlines several key reasons why explainable black boxes should be avoided in high-stakes decisions, identifies challenges to interpretable machine learning, and provides several example applications where interpretable models could potentially replace black box models in criminal justice, healthcare and computer vision.

Recap - Feature learning

- ❖ Feature extractor (Conv_layer) is learned by computer itself
- ❖ Uses multiple layers to progressively extract higher-level features from raw input (e.g. image)
- ❖ Higher-level features and concepts are defined in terms of lower-level ones
- ❖ “Hierarchical” Feature Learning
 - **Deep Learning**

Lower-level features (e.g. edges) $\Rightarrow$ Higher-level features (e.g. cat nose/ whiskers?)



2. Blackbox nature of deep learning model

Rapid growth (e.g. wide range of applications) but lack of understanding

Input > “blackbox” > Output

- We are now “blackbox” and back to traditional model again
- Will deep learning involve making high-stakes decisions?

Solutions:

Combine use of traditional and deep learning models

Add rules/instructions to deep learning model

...

(Understand the results of the model with scientific basis)

From deep learning to deep reasoning?

Deep Reasoning: Is this the Next Era of AI?

Deep reasoning is said to bring us all one step closer to artificial general intelligence. It is said to be the next level beyond deep learning.

Deep Reasoning - The Science of Machine Learning

Deep Reasoning combines deep learning with reasoning for solving complex tasks. It is a critical step toward progress in achieving Artificial General ...

Abstraction, Reasoning and Deep Learning: A Study of the "Look and Say" Sequence

Wlodek W. Zadrozny^{1,2},
wzadroz@unc.edu

¹ College of Computing, University of North Carolina at Charlotte

² School of Data Science, University of North Carolina at Charlotte

version 2.0 January 2022

Abstract

The ability to abstract, count, and use System 2 reasoning are well-known manifestations of intelligence and understanding. In this paper, we argue, using the example of the "Look and Say" puzzle, that although deep neural networks can exhibit high 'competence' (as measured by accuracy) when trained on large data sets (2 million examples in our case), they do not show any sign on the deeper understanding of the problem, or what D. Dennett calls 'comprehension'. We report on two sets of experiments: first, computing the next element of the sequence, and, then, the previous element. We view both problems as building a translator from one set of tokens to another. We apply both standard LSTMs and Transformer/Attention-based neural networks, using publicly available machine translation software. We observe that despite the amazing accuracy, the performance of the trained programs on the actual L&S sequence is bad, and shows no understanding of the principles behind the sequences. The ramifications of this finding include: (1) from the cognitive science perspective, we argue that we need better mathematical models of abstraction; (2) the universality of neural networks should be re-examined for functions acting on discrete data sets; (3) we hypothesize topology can provide a definition of without the reference to the concept of distance.

Keywords: deep learning, Look and Say puzzle, cognitive System 2, reasoning, universality of neural networks, topology, TDA.

From Deep Learning to Deep Reasoning

Authors: Truyen Tran, Vuong Le, Hung Le, Thao M. Le [Authors info & Claims](#)

KDD '21: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining • August 2021 • Pages 4076–4077 • <https://doi.org/10.1145/3447548.3470803>

Online: 14 August 2021 [Publication History](#)

1 299

Get Access

ABSTRACT

The rise of big data and big compute has brought modern neural networks to many walks of digital life, thanks to the relative ease of constructing large models that scale to the real world. Current successes of Transformers and self-supervised pretraining on massive data have led some to believe that deep neural networks will be able to do almost everything once we have sufficient data and computational resources. However, neural networks are fast to exploit surface statistics but fail miserably to generalize to novel combinations. This is because they are not designed for deliberate reasoning -- the capacity to deliberately deduce new knowledge out of the contextualized data. This tutorial reviews recent developments to extend the capacity of neural

<https://dl.acm.org/doi/abs/10.1145/3447548.3470803>

<https://arxiv.org/abs/2109.12755>

Thank you